

MA-224 – Lecture

Discrete Mathematics 2

Regular expressions and automata

Andreas Prinz

Sverre Lunøe-Nielsen



Today

- Languages
- Regular expressions
- Automata
- NFA and regex
- Minimize automata
- Summary

Languages and alphabets

- Languages are based on an alphabet A (arbitrary set).
- All strings over A are called A^* (recursive definition)
 - $\varepsilon \in A^*$ - The empty string is a string.
 - $s \in A^* \wedge l \in A \Rightarrow s^\circ l \in A^*$ - If s is a string and l is a letter, then $s^\circ l$ (appending l at the end of s) is a string.
 - There are no other strings (implied).
- Any language L is a subset of A^* .
 - $L \subseteq A^*$ - note the dependence on the alphabet!
- This means, it can be decided for each specific string, if it is in the language or not.

Language operations

- Empty language: $\emptyset$
- Language with only empty word: $\varepsilon = \{\varepsilon\}$
- Union $L_1 \cup L_2$: all words in L_1 and L_2
- Concatenation $L_1 L_2$: all words $l_1 + l_2$ where $l_1 \in L_1$ and $l_2 \in L_2$
- Iterated concatenation L^i : $L^0 = \{\varepsilon\} = \varepsilon$, $L^1 = L$, $L^{n+1} = L^n + L$
- Kleene closure L^* : $\bigcup_{i=0}^{\infty} L^i$ – zero or more concatenations of L
- Positive closure L^+ : $\bigcup_{i=1}^{\infty} L^i$ – one or more concatenations of L

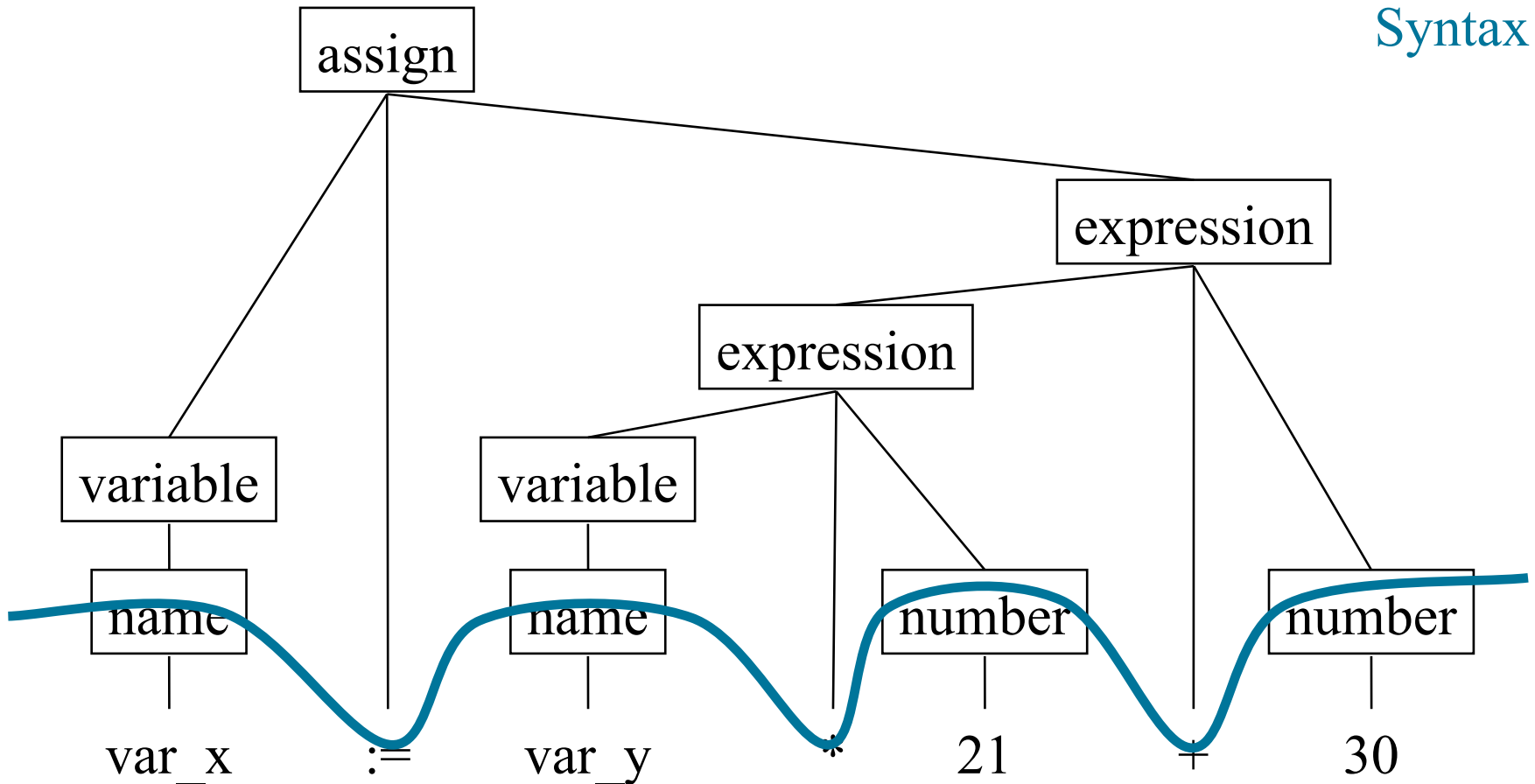
What are words? ☾ Lexis

- Low-level alphabet: characters (ASCII)
 - Lexemes = groups of characters that are considered a unit
 - Tokens = collection (set) of lexemes of the same kind
- Sample lexical items
 - ID `[a-zA-Z_][a-zA-Z_0-9]*`
 - Keyword `class | if | in | ...`
 - Operators `> | < | = | > = | < = | < > | ...`
- Rules
 - Always take the longest possible string
 - Ignore whitespaces



Syntax tree – part of lexis

Syntax



Lexis

Tasks for the lexer

- Strip the input from irrelevant information (whitespaces, comments)
- Correlate error messages to input lines
- Why do we want a lexer?
 - Lexer is simpler and more efficient because it is regular
 - A separate lexer enhances portability (alphabets)
- Lexer terms
 - token – this is the language the lexer accepts (typically complete!)
 - pattern – this is a rule for describing a token (matching rule)
 - lexeme – this is one special instance of a token in the input text
- How to construct a lexer? – use a tool, e.g. lex
 - Make a language with alternatives from patterns for all tokens
 - Create the related NFA and DFA with all the final states

Sample tokens

<i>RegExpr</i>	<i>TOKEN</i>	<i>Value</i>
whitespace: (tab nl sp)+	-	-
if	if	-
else	else	-
elsif	elsif	-
id: [a-zA-Z][a-zA-Z0-9]*	id	string
num: [0-9]+(.[0-9]+)?(E[+-]?[0-9]+)?	num	val
<, <=	relop	lt, le
=, <>	relop	eq, ne
>, >=	relop	gt, ge

- take the longest matching token and have precedence if same length
- make sure to have a catchall, i.e. there are no errors in the lexer



Today

- Languages
- Regular expressions
- Automata
- NFA and regex
- Minimize automata
- Summary



Regular expressions (math)

- A language using the following parts is a regular language
- Basic elements (Recursion base)
 - $\emptyset$: denotes the empty language
 - ε : (the empty string) denotes the language with only the empty string: $\{\varepsilon\}$
 - a : (if $a \in A$) denotes the language with only the string a : $\{a\}$
- Operators (Recursion step)
 - $(r)|(s)$: alternative denotes $L(r) \cup L(s)$
 - $(r)(s)$: concatenation denotes $L(r)L(s)$
 - $(r)^*$: closure denotes $L(r)^*$

Regular expressions (CS)

- The following elements are used in patterns.
- Basic elements
 - `.`: wildcard (any character)
 - `[...]`: single character – matches one of the characters in the bracket, e.g. `[abc]`
 - `[.-.]`: single character from range – matches one of the characters in the range, e.g. `[a-h]`
 - `[^...]`: inverse single character – matches one of the characters not included in the bracket
 - Posix predefined character classes:
`[:alnum:]`, `[:alpha:]`, `[:blank:]`, `[:cntrl:]`, `[:digit:]`,
`[:graph:]`, `[:lower:]`, `[:print:]`, `[:punct:]`,
`[:space:]`, `[:upper:]`, `[:xdigit:]`

Regular expressions (CS)

- The following operators are used in patterns.
- Operators
 - |: alternative
 - (): parenthesis for grouping
 - ?: optional (0..1)
 - *: Kleene closure (0..)
 - +: positive closure (1..)
 - {n}: exactly n
 - {n,}: at least n (n..)
 - {,m}: at most m (..m)
 - {n,m}: between n and m (n..m)



Beyond regular languages (CS)

- Regular expressions often feature elements that are beyond regular languages as follows.
- `^` and `$` are used to indicate start and end of line
- Sub-expressions can be grouped with `()` and then referenced back using `\1` or `\2`
Example `(.+) \1` matches `HalloHallo` and `HaHa`



Translate regular expressions

- Equalities
 - $a? = (a|\epsilon)$
 - $a+ = aa^*$
 - $a^* = a+|\epsilon$
 - $a\{n\} = a...a$ (n repetitions)
 - $a\{n,\} = a\{n\}a^*$
 - $a\{,m\} = (a?)\{m\}$
 - $a\{n,m\} = a\{n\}a\{,m-n\}$
- Avoid parentheses: precedence and associativity
 - $*$ highest, left
 - concatenation next, left
 - $|$ lowest, left



Examples

• Are these expressions equal?

$$\bullet (a|b)(a|b) = aa|ab|ba|bb$$

$$\bullet ab|cd^* = a(b|c)(d^*) \quad (| * \circ)$$

$$\bullet ab|cd^* = (a(b|c)d)^* \quad (| \circ *)$$

$$\bullet ab|cd^* = ((ab)|(cd))^* \quad (\circ | *)$$

$$\bullet ab|cd^* = (ab)|((cd)^*) \quad (\circ * |)$$

$$\bullet ab|cd^* = a(b|c)(d^*) \quad (* | \circ)$$

$$\bullet ab|cd^* = (ab)|(c(d^*)) \quad (* \circ |)$$

$$\bullet (a|b)^* = (a^*b^*)^* = (a?b?)^*$$

$$\bullet (a^*b^*)^* = (a^*b)^*$$

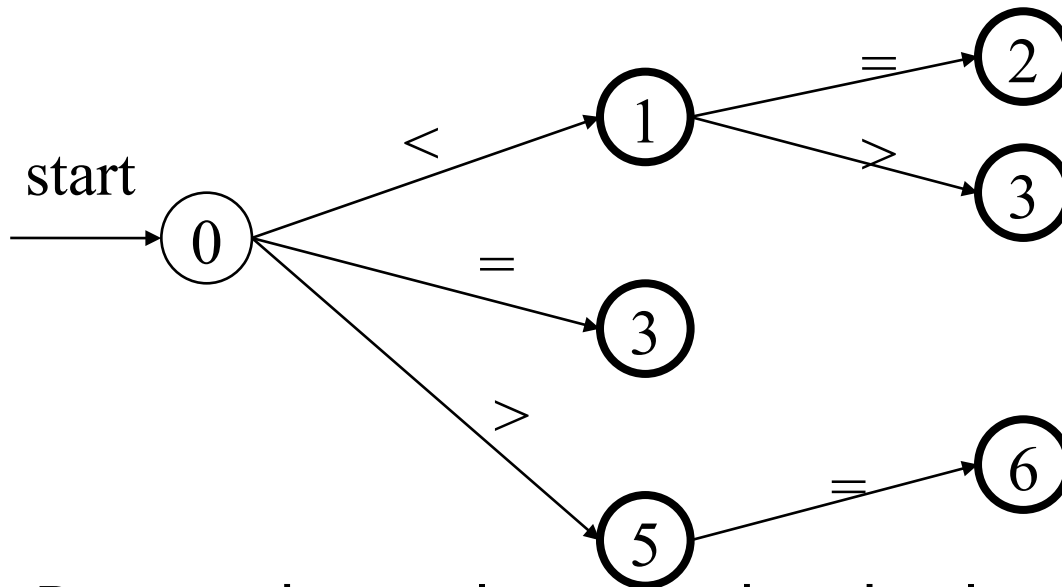


Today

- Languages
- Regular expressions
- Automata
- NFA and regex
- Minimize automata
- Summary

Lexical analysis of logical ops

- Finite automata for analysis (generated)
- transitions triggered by characters read

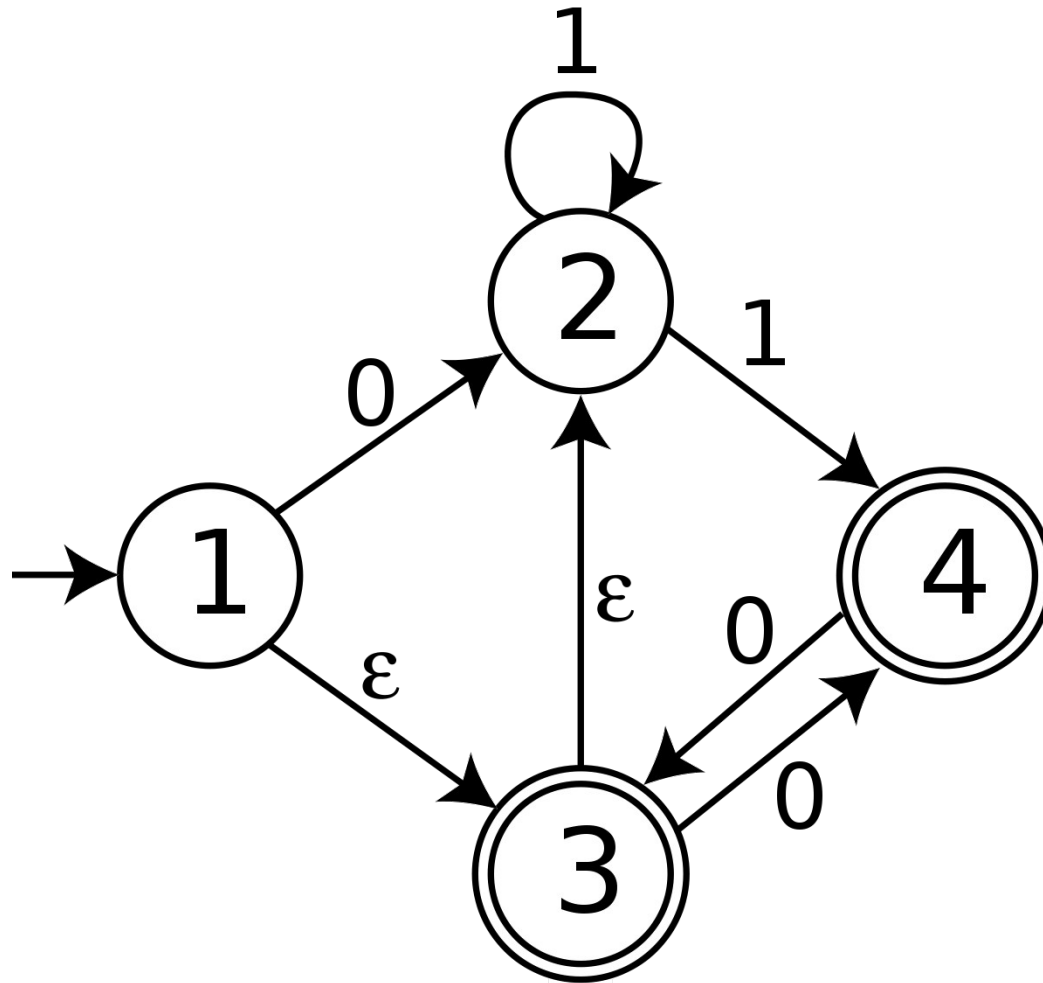


- Remember: always take the longest possible string

Nondeterministic finite automata

- nondeterministic finite automaton (NFA)
(S, A, T, s, F)
 - States S , Alphabet A , start state $s \in S$, final (accepting) states $F \subseteq S$
 - Transition function $T: S \times (A \cup \{\varepsilon\}) \rightarrow P(S)$
- similar to transition graph, but we can have two equal transition labels from one single state and empty string as label
- accept a string, if there is a path to a final state

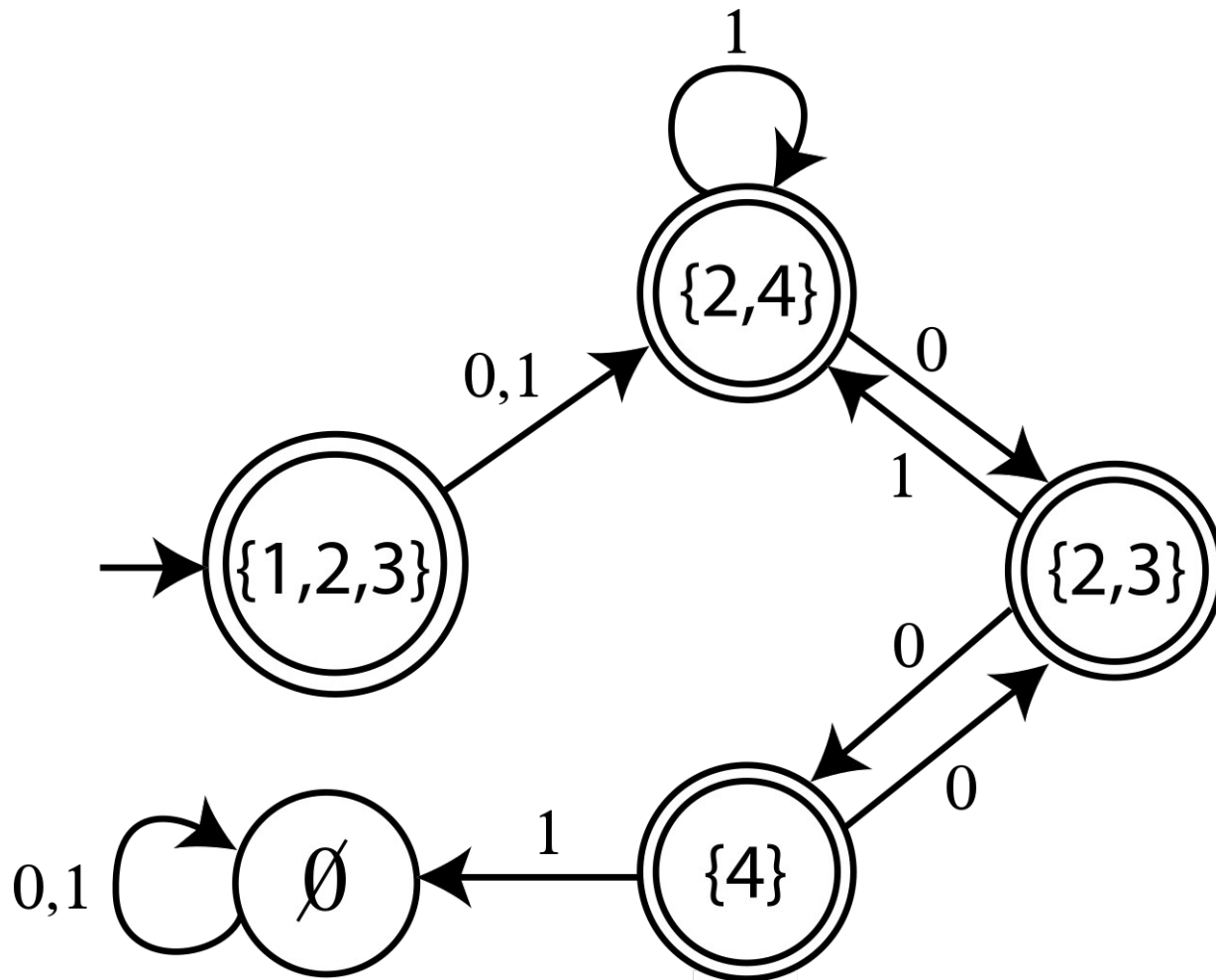
Example NFA



Deterministic finite automata

- deterministic finite automaton (NFA)
(S, A, T, s, F)
 - States S , Alphabet A , start state $s \in S$, final (accepting) states $F \subseteq S$
 - Transition function $T: S \times A \rightarrow S$
- accept a string, if there is a path to a final state
- as NFA, but Transition function is deterministic

Example DFA

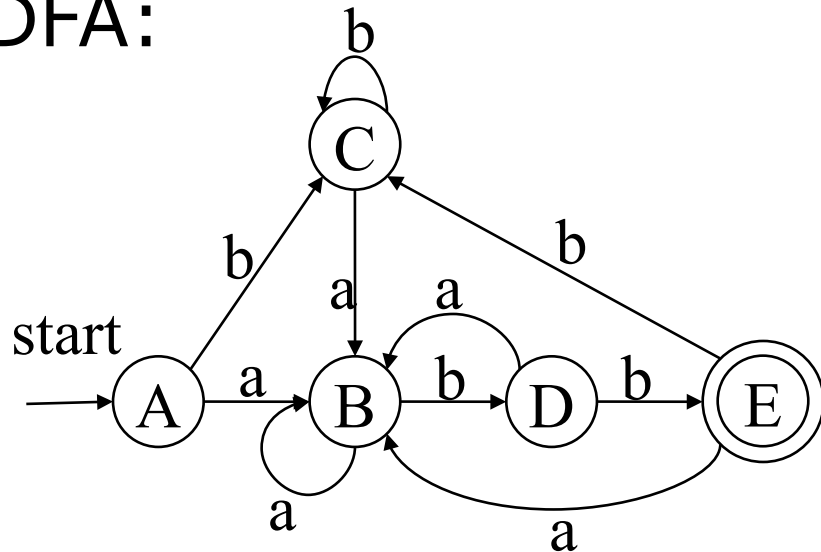


Language definitions

- There are two possible ways to define a language.
- *Generation* defines the language by giving a generator process to create words.
- *Recognition* defines a language by giving a decision procedure whether an element is in the language or not.

Language, Grammar, DFA

DFA:



Language: $[ab]^*abb$

Grammar:

$A ::= B a b b .$

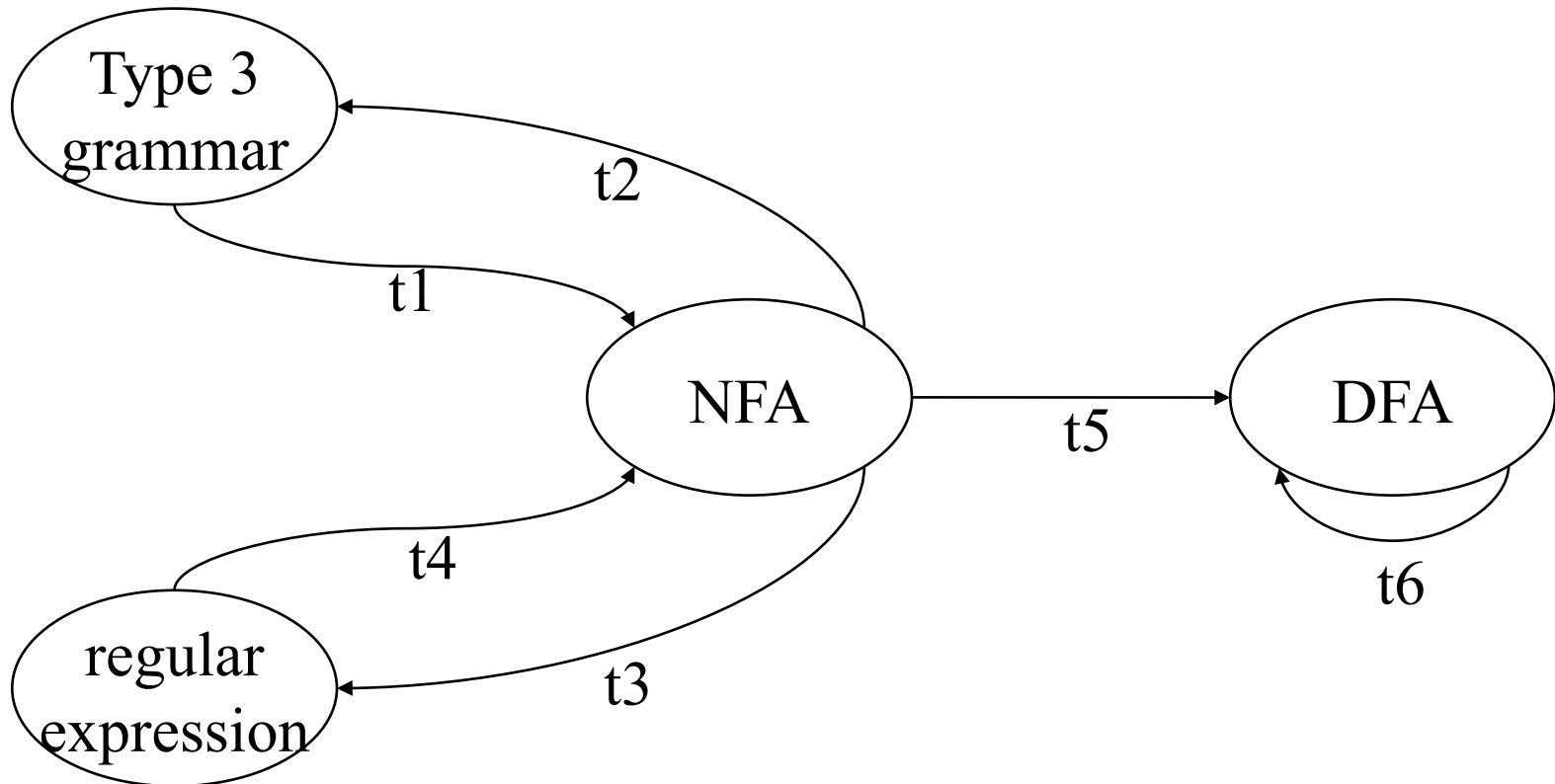
$B ::= a B \mid b B \mid \varepsilon .$

sample inputs: abb, **aabb**, ababb, babb,
abbabb, abbbbabb, abab

Experiment: check abb, abab, abbabb

What we do now

t = translate





Regular grammars (repetition)

- A grammar is a structure (T, NT, s, R) with
 - T is an arbitrary set of terminal symbols
 - NT is an arbitrary set of nonterminal symbols (meta-symbols) disjoint with T
 - s is the start symbol, $s \in NT$
 - R is a set of Rules, where each rule is a pair (l, r) , where $l, r \in (T \cup NT)^*$
- Type-3 grammars (regular grammars/languages).
 - Right-regular: rules of the form $A \rightarrow \varepsilon$, $A \rightarrow t$, $A \rightarrow tB$, $A \rightarrow B$, with $A, B \in NT$; $t \in T$.

Regular grammars & languages

- Languages generated by regular expressions = Languages generated by regular grammars
 1. We can prove that any regular grammar has an NFA and vice versa.
 2. We can prove that any regular expression has an NFA and vice versa.



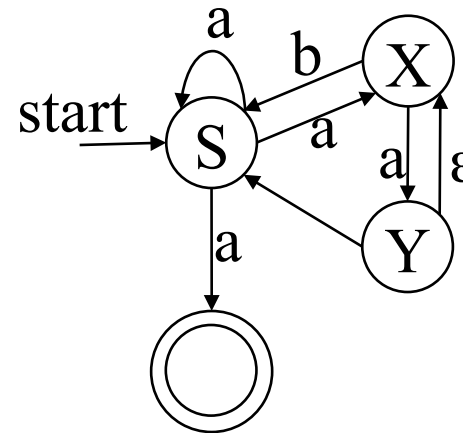
Proofsketch Part 1 forward

- We translate the grammar into an NFA as follows:
 - 1) all NTs are states
 - 2) all rules are transitions:
 - $A \rightarrow \varepsilon$: create a transition from A to an unlabeled final state with label ε
 - $A \rightarrow t$: create a transition from A to an unlabeled final state with label t
 - $A \rightarrow tB$: create a transition from A to B with label t
 - $A \rightarrow B$: create a transition from A to B with label ε



Example grammar to NFA

- $S ::= aS \mid aX \mid a$
- $X ::= bS \mid aY$
- $Y ::= bS \mid X$



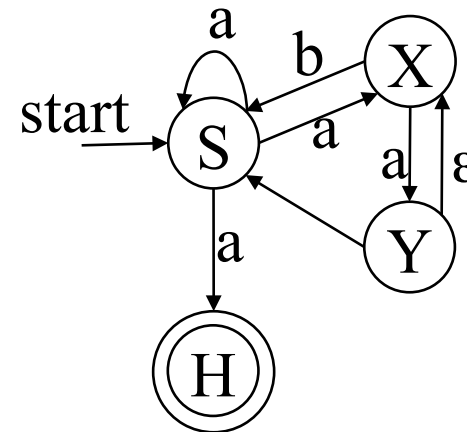


Proofsketch Part 1 backward

- We translate the NFA into a grammar as follows:
- 1) all states are NTs
- 2) all transitions are rules:
 - A transition from A to B with label t: $A \rightarrow tB$
 - A transition from A to B with label ε : $A \rightarrow B$
 - A final state A: $A \rightarrow \varepsilon$



Example NFA to grammar



- $S ::= aS \mid aX \mid aH$
- $X ::= bS \mid aY$
- $Y ::= bS \mid X$
- $H ::= \epsilon$

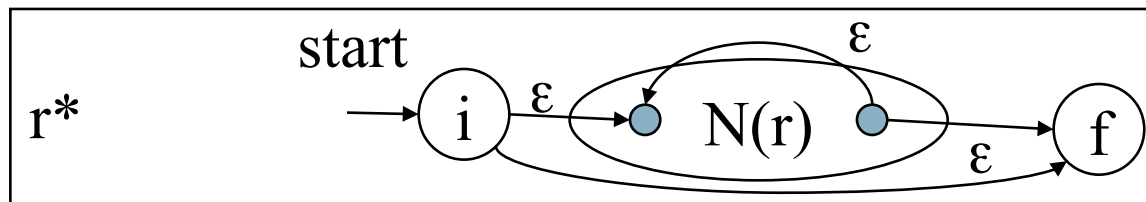
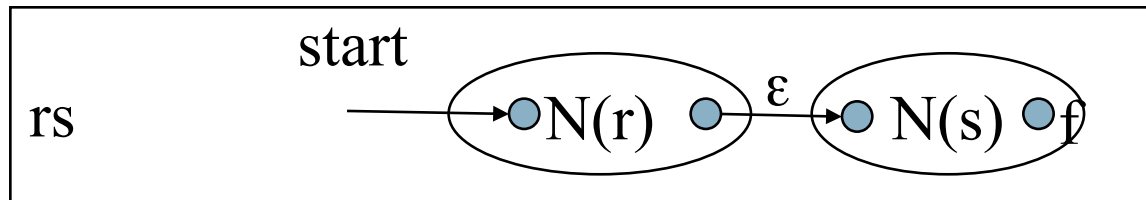
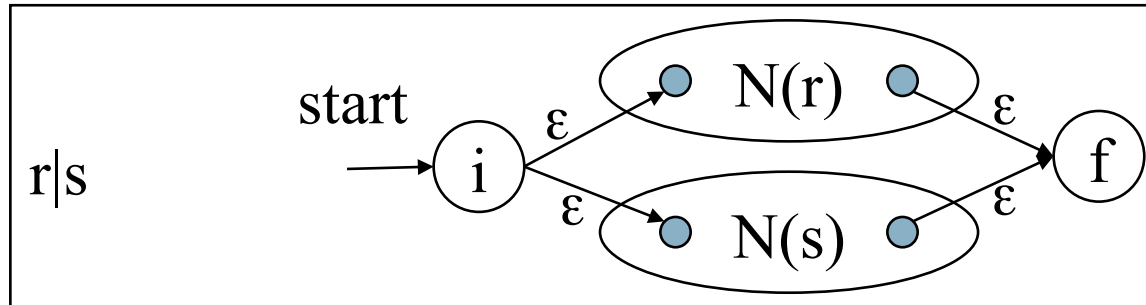
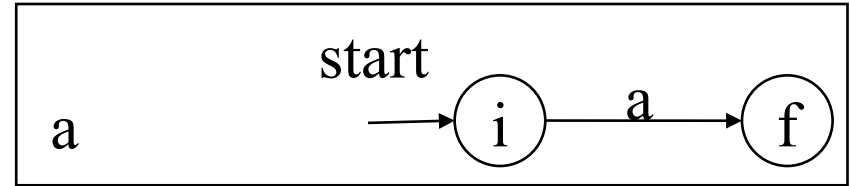
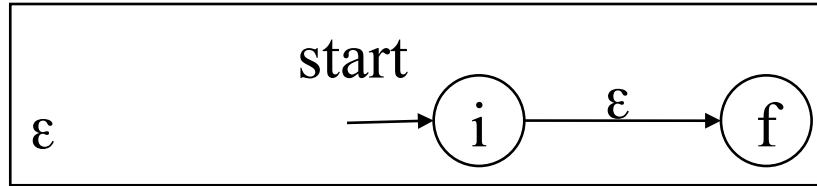


Today

- Languages
- Regular expressions
- Automata
- NFA and regex
- Minimize automata
- Summary



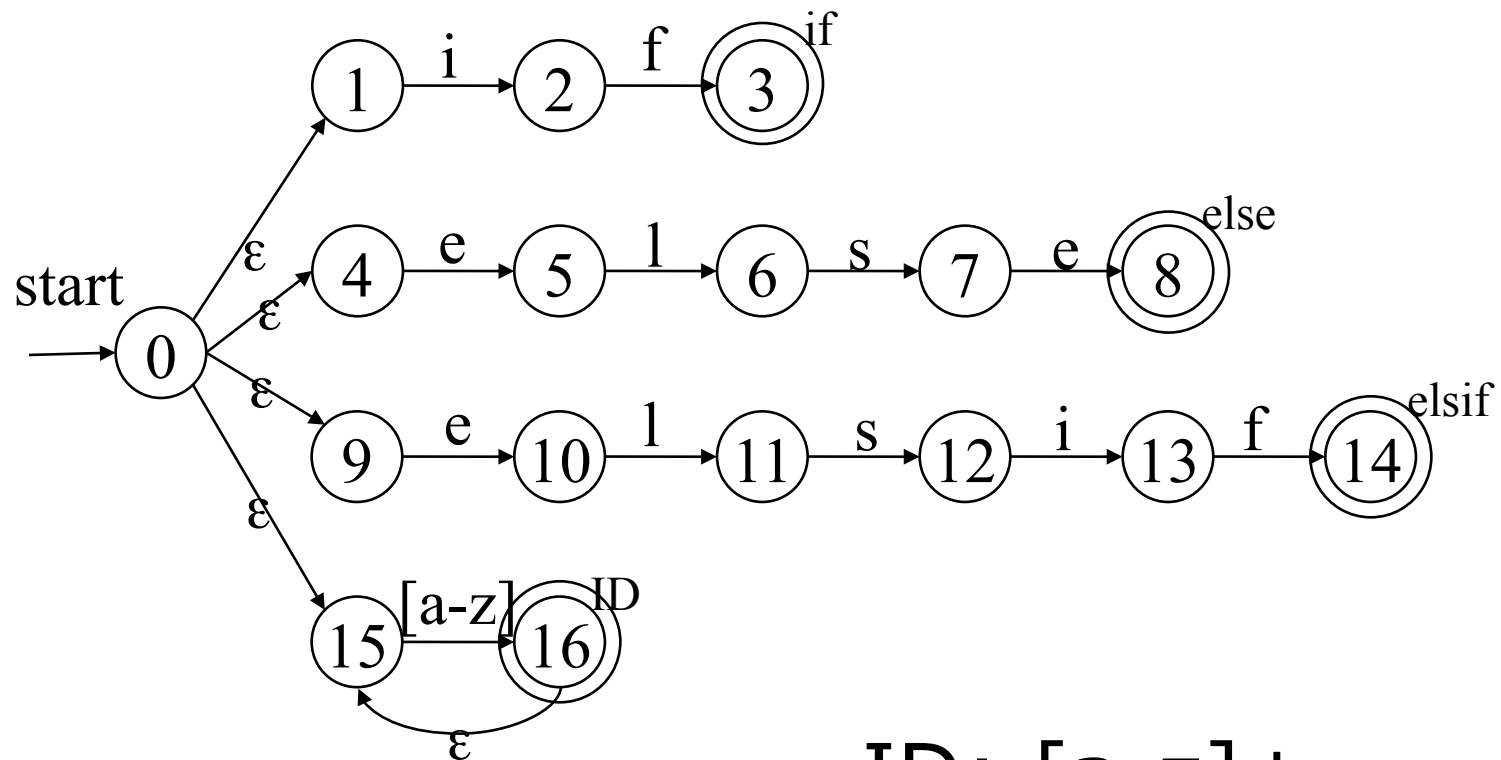
Regular expression $\rightarrow$ NFA



(r): use N(r)



Example: NFA for ID, if, else, elsif



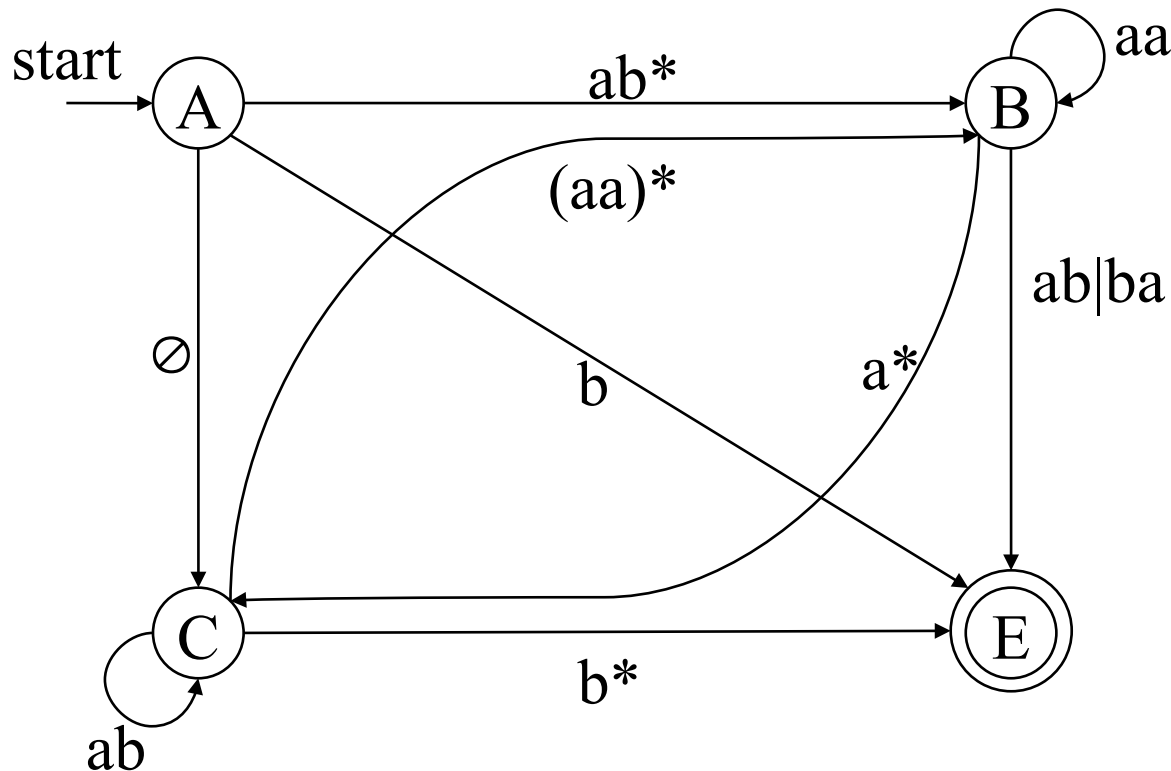
ID: $[a-z]^+$



Translating NFA to regex

- Algorithm idea:
 - Use extended NFA with regex as labels
 - Normalize NFA by inserting init and final
 - Remove states by introducing the regex for the paths of their edges
 - Continue until all states are removed
 - The remaining regex is the collected regex of the NFA

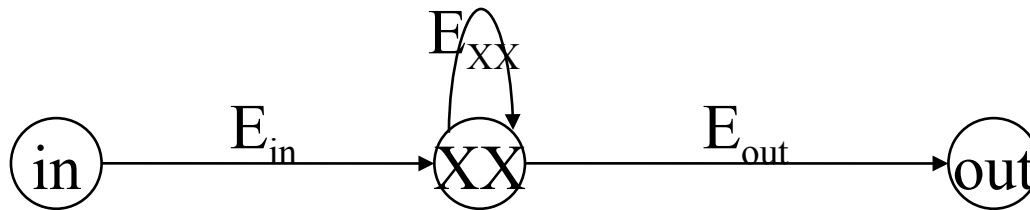
Example: extended NFA with regex



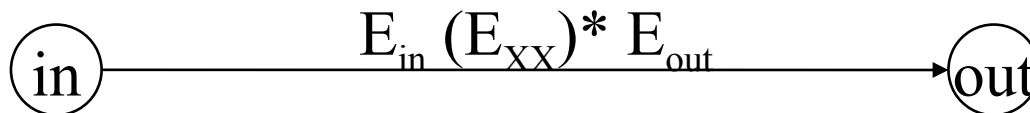
- $\emptyset$ means the path is unavailable

Removing states

Replace all connections of the form



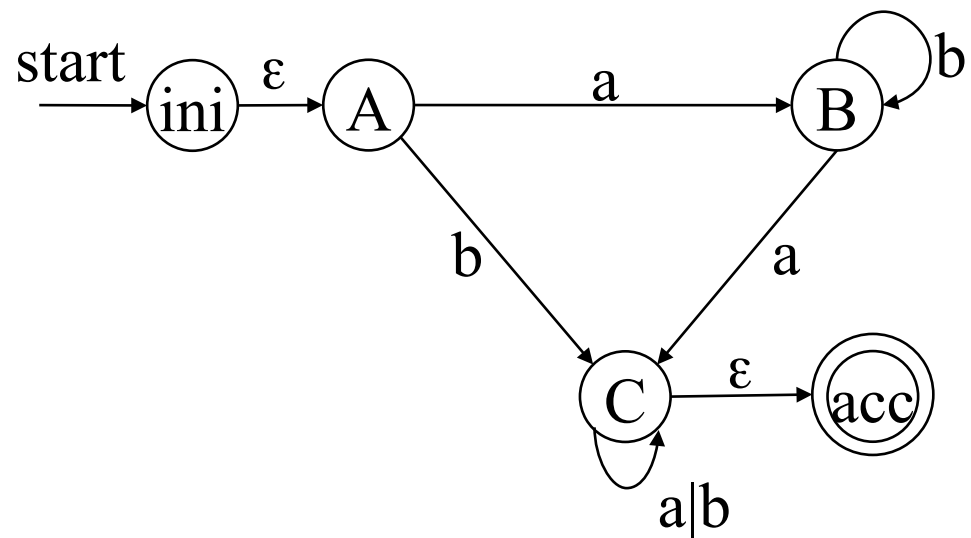
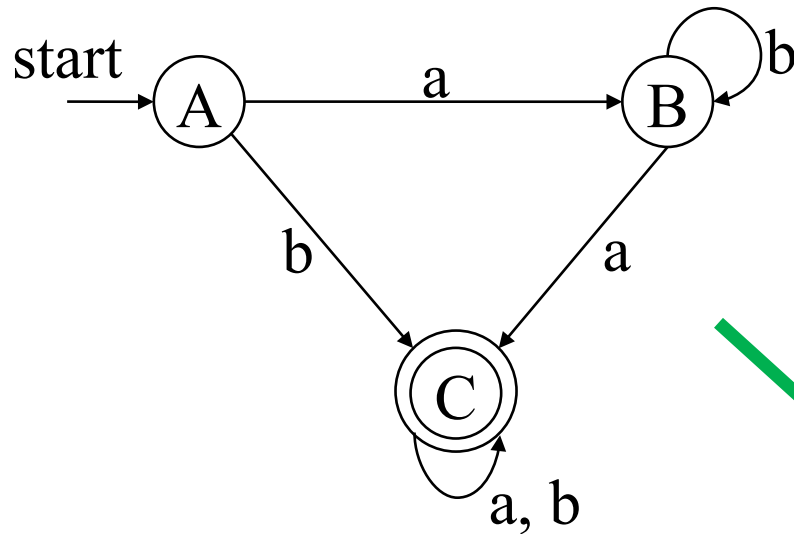
by connections of the form



Then remove the state XX .

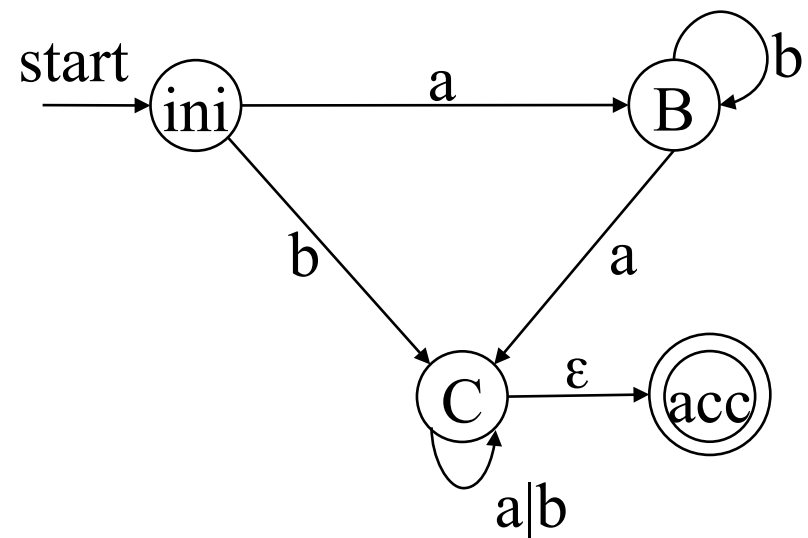
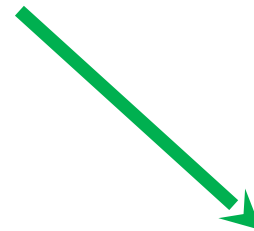
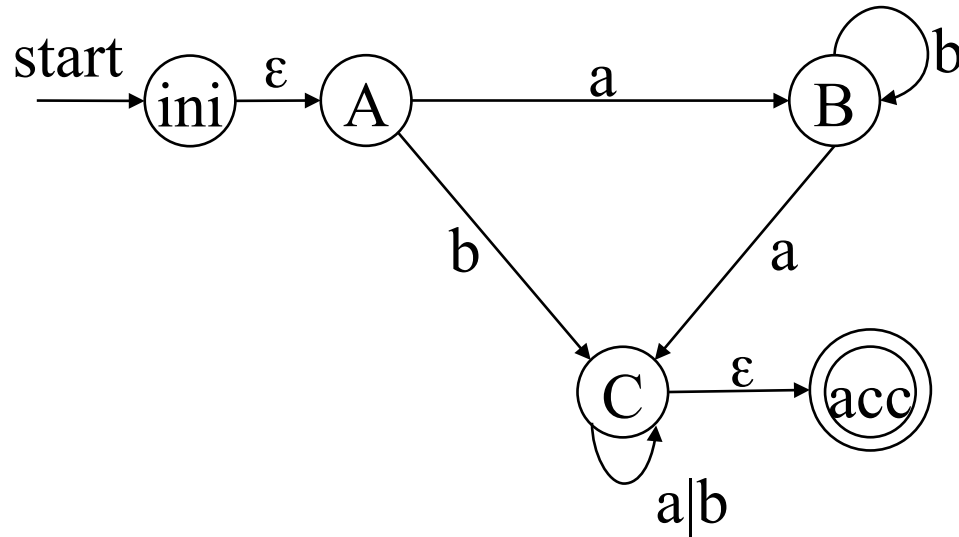


Example: NFA to regex: normalize



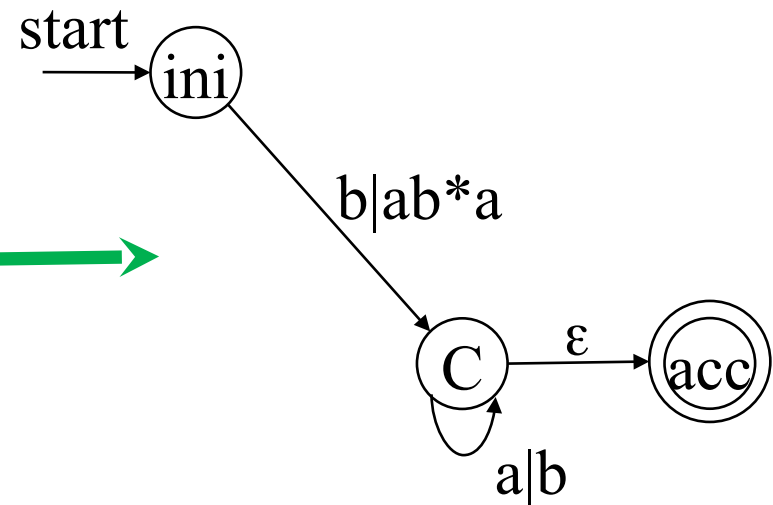
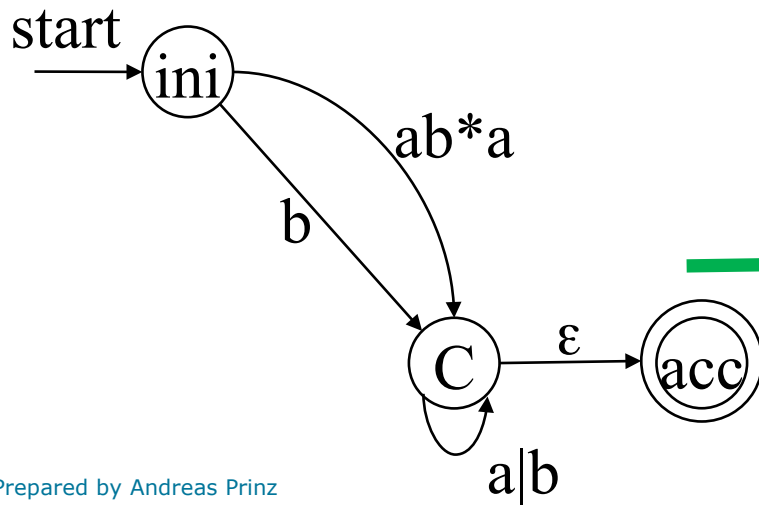
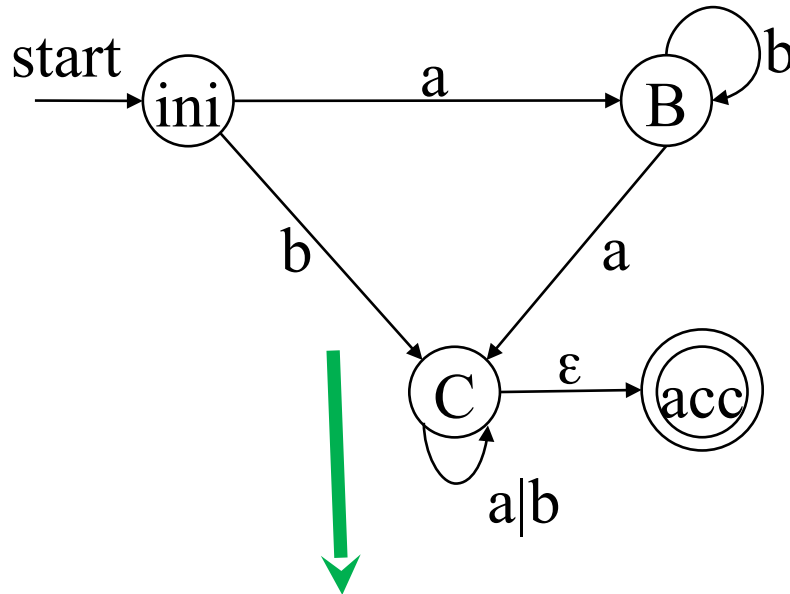


Example: Removing state A

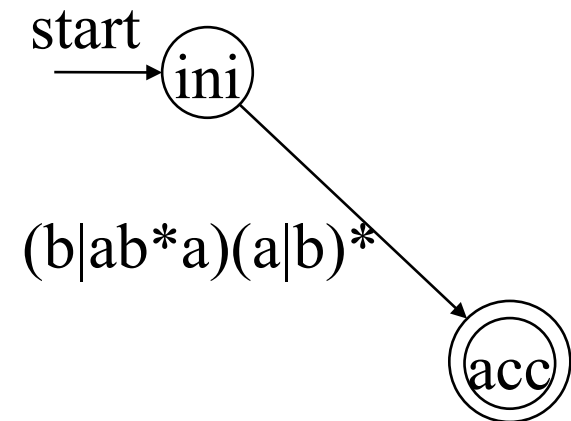
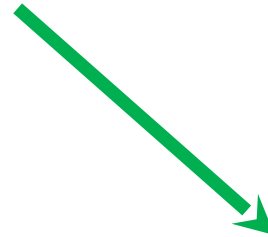
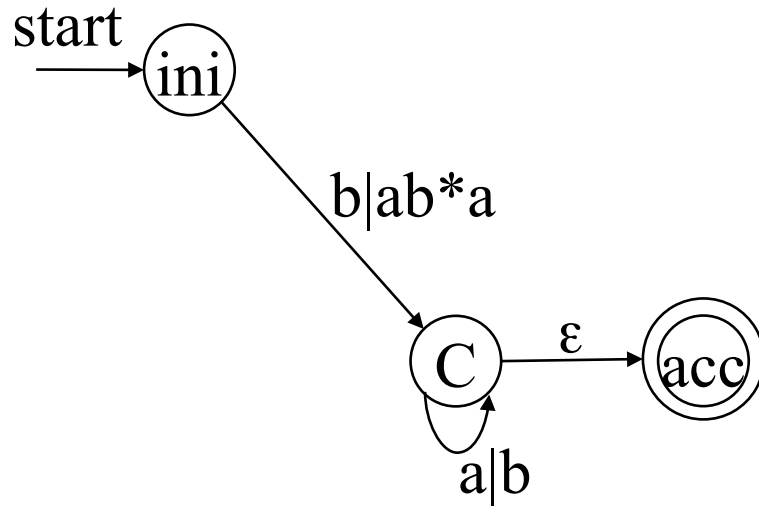




Example: Removing state B



Example: Removing state C





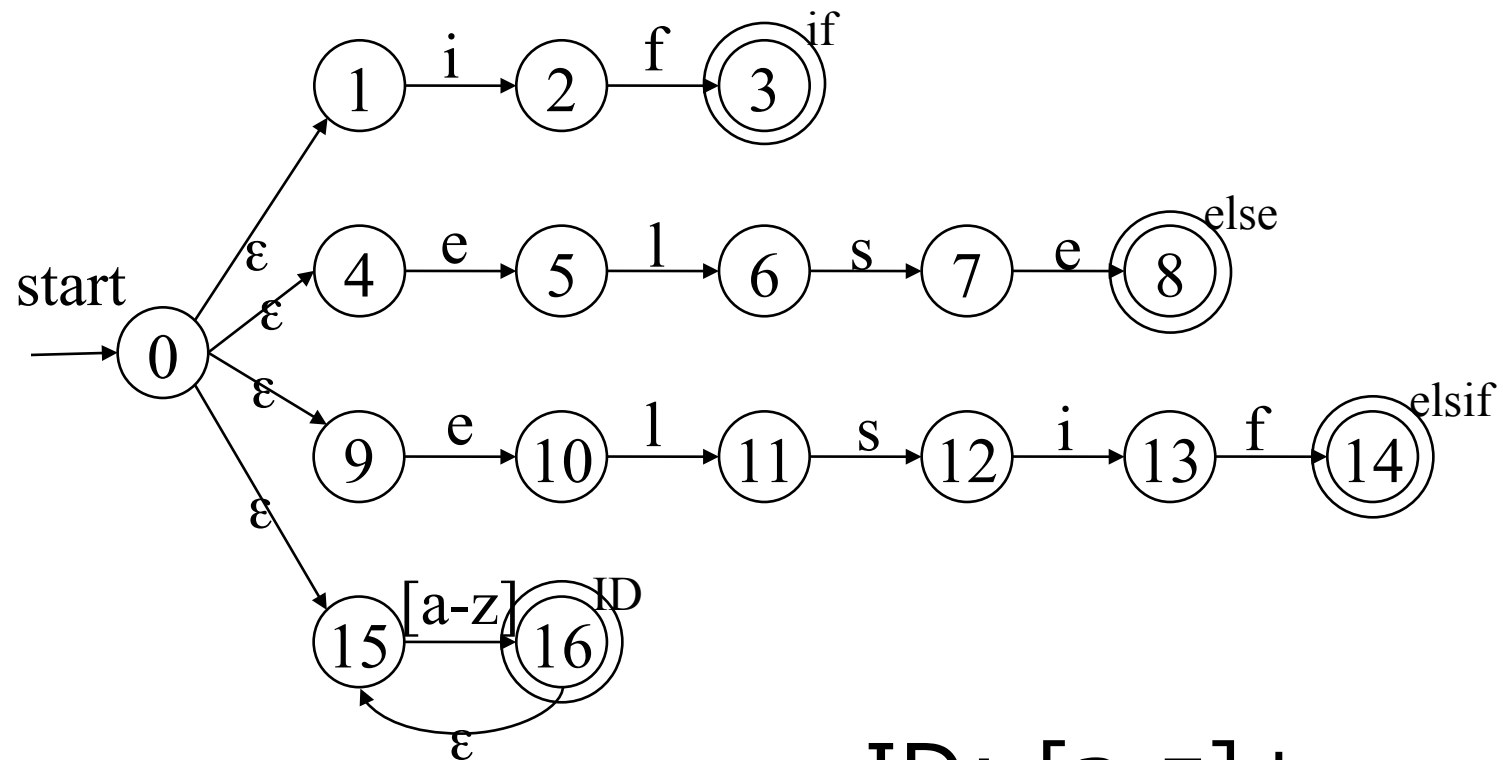
Conversion NFA $\rightarrow$ DFA

- NFA and DFA languages are equivalent
- transformation algorithm: consider sets of DFA-states as single NFA-state
 - consider states as equivalent if they are connected via ε -Transitions
 - next state (set) after input a is the state consisting of all states that are reachable via a

Conversion algorithm

1. Create the NFA start state as the set consisting of the DFA start state.
2. Expand the NFA start state by adding all states reachable via ε -Transitions.
3. Take one state of the current NFA and one symbol that has a transition in any of the DFA states of the NFA state. Create a transition from the state with the symbol into the expanded state of all resulting transitions for this symbol from the DFA states in the NFA state.
4. Continue until no change.

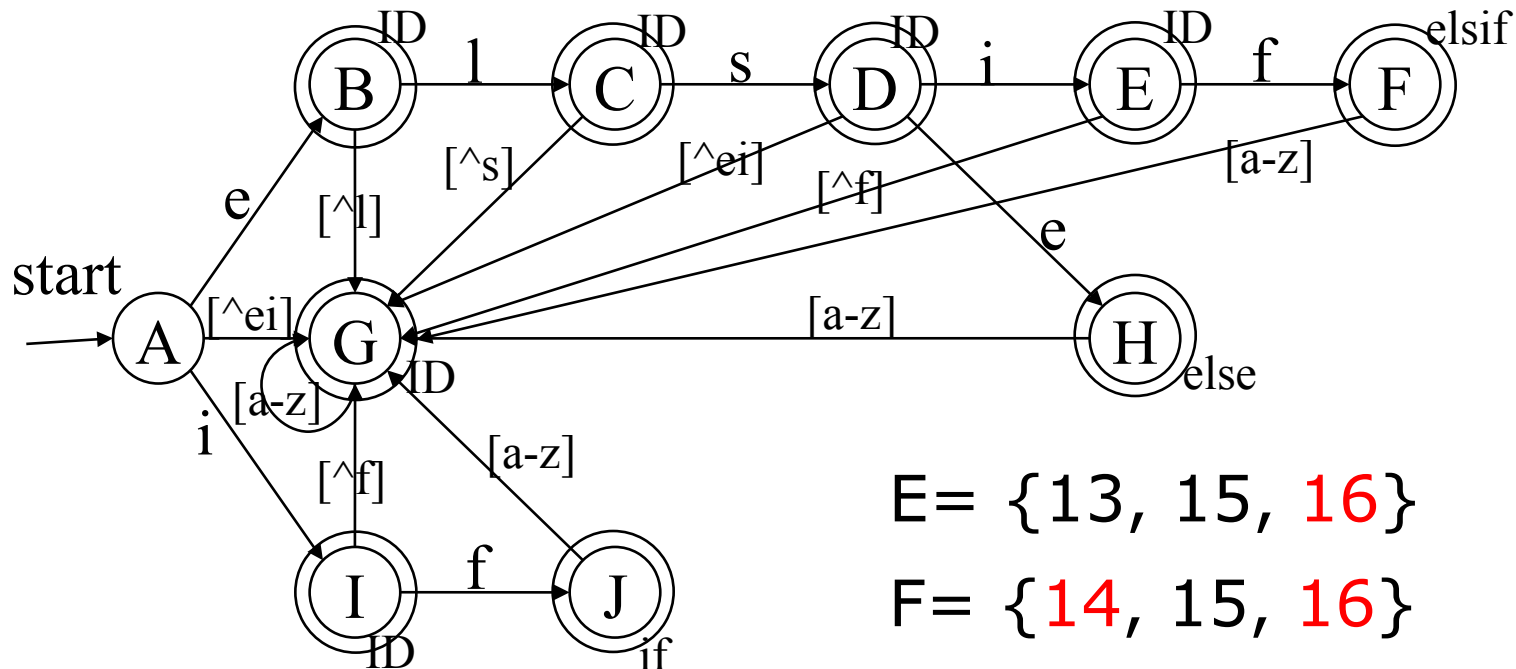
Example: NFA for ID, if, else, elsif



ID: $[a-z]^+$



Example: DFA for ID, if, else, elsif



A = {0, 1, 4, 9, 15}

B = {5, 10, 15, 16}

C = {6, 11, 15, 16}

D = {7, 12, 15, 16}

E = {13, 15, 16}

F = {14, 15, 16}

G = {15, 16}

H = {8, 15, 16}

I = {2, 15, 16}

J = {3, 15, 16}



Today

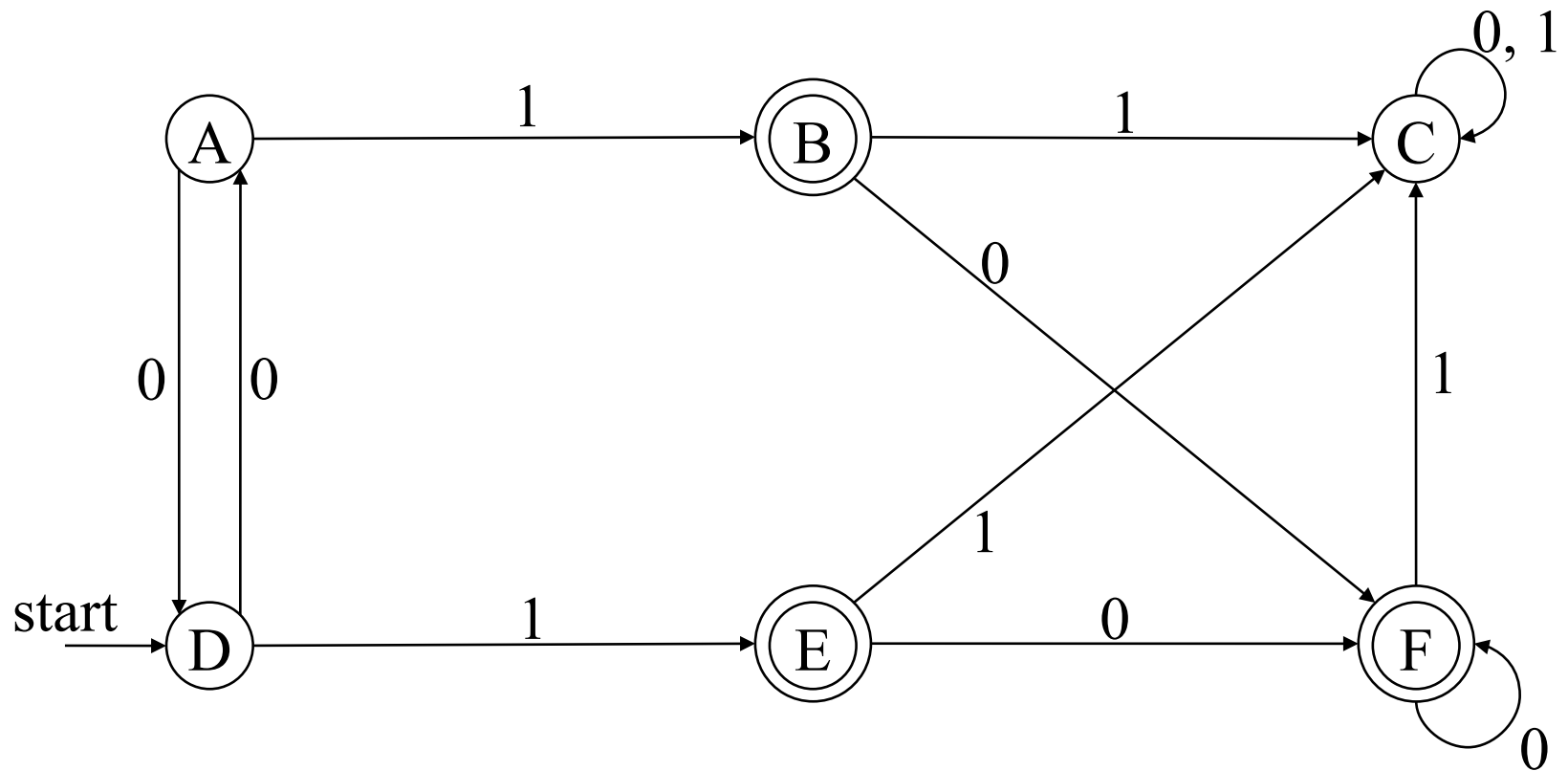
- Languages
- Regular expressions
- Automata
- NFA and regex
- Minimize automata
- Summary

Minimize DFA

- DFA minimization stands for converting a given DFA to its equivalent DFA with minimum number of states. The following algorithm will do.
 1. Remove unreachable states.
 2. Divide the set of states into two sets: final states and non-final states. This partition is called P_0 .
 3. Find P_{k+1} recursively by partitioning the sets of P_k . In each set of P_k , separate the states that are distinguishable, thus getting P_{k+1} .
 4. Stop when $P_{k+1} = P_k$ (No change in partition)
 5. The sets of the last P_k are the states of the minimal DFA and have their transitions.
- Two states s_i and s_j are distinguishable in partition P_{k+1} if for any input symbol a , their transitions $T(s_i, a)$ and $T(s_j, a)$ are in different sets in partition P_k .



Example: minimizing DFA



$$P_0 = \{\{A, C, D\}, \{B, E, F\}\}$$



Step 3.1a: partition $\{A, C, D\}$

- Distinguish A and C?
 $T(A,0)=D$, $T(C,0)=C$ with C and D being in the same partition.
 $T(A,1)=B$, $T(C,1)=C$ with B and D being in different partitions.
So we can distinguish A and C.
- Distinguish A and D?
 $T(A,0)=D$, $T(D,0)=A$ with A and D being in the same partition.
 $T(A,1)=B$, $T(D,1)=E$ with B and E being in the same partition.
So we cannot distinguish A and D.
- Distinguish C and D?
 $T(C,0)=C$, $T(D,0)=A$ with A and C being in the same partition.
 $T(C,1)=C$, $T(D,1)=E$ with C and E being in different partitions.
So we can distinguish C and D.
- The new partitioning is now $\{A, D\}, \{C\}$.



Step 3.1b: partition $\{B, E, F\}$

- Distinguish B and E?
 $T(B,0)=T(E,0)=F$ and $T(B,1)=T(E,1)=C$
So we cannot distinguish B and E.
- Distinguish B and F?
 $T(B,0)=T(F,0)=F$ and $T(B,1)=T(F,1)=C$
So we cannot distinguish B and F.
- Distinguish E and F?
As we cannot distinguish B and E and cannot distinguish B and F, we cannot distinguish E and F either.
- The partitioning does not change for $\{B, E, F\}$.

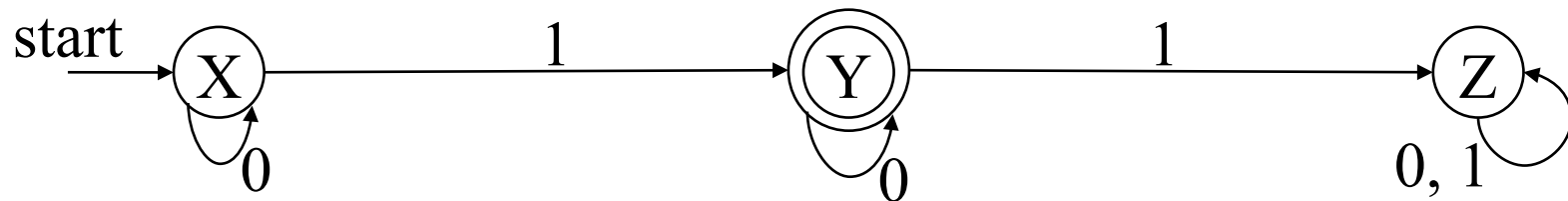


Step 3.2

- We cannot partition $\{C\}$ further as it is only one state.
- Distinguish A and D?
 $T(A,0)=D$, $T(D,0)=A$ with A and D being in the same partition.
 $T(A,1)=B$, $T(D,1)=E$ with B and E being in the same partition.
So we cannot distinguish A and D.
- This means the partition $\{A,D\}$ is unchanged.
- Distinguish B and E? ☾ $T(B,0)=T(E,0)=F$ and $T(B,1)=T(E,1)=C$
So we cannot distinguish B and E.
- Distinguish B and F? ☾ $T(B,0)=T(F,0)=F$ and $T(B,1)=T(F,1)=C$
So we cannot distinguish B and F.
- Distinguish E and F? As before, we cannot distinguish E and F.
- The partitioning does not change for $\{B,E,F\}$.



Example: minimal DFA



$X = \{A, D\}, Y = \{B, E, F\}, Z = \{C\}$

Summary

- Regular expressions and regular grammars and NFA and DFA define the same set of languages – they are equivalent.
- Regular expressions can be translated to NFA & back.
- Regular grammars can be translated to NFA & back.
- NFA can be translated into DFA.
- DFA (and NFA) can be minimized.
- Programming languages use regular expressions and their DFAs to define their lexical structure.
- Regular expressions can be found quickly by DFAs.